

스마트 공조기

박준모 박지현

I 목차

Chapter 1. 서론

Chapter 2. block diagram & FSM

Chapter 3. source code

Chapter 4. data 분석

Chapter 5. 동작 동영상

Chapter 6. 결론

서론

Basys3 보드, DHT11(온습도 센서), DS1302(RTC)를 이용한 스마트 공조기

핵심 동작

1) DHT11(온습도 센서) 기능

- 온도.습도값 FND, UART 출력

2) DS1302(RTC) 기능

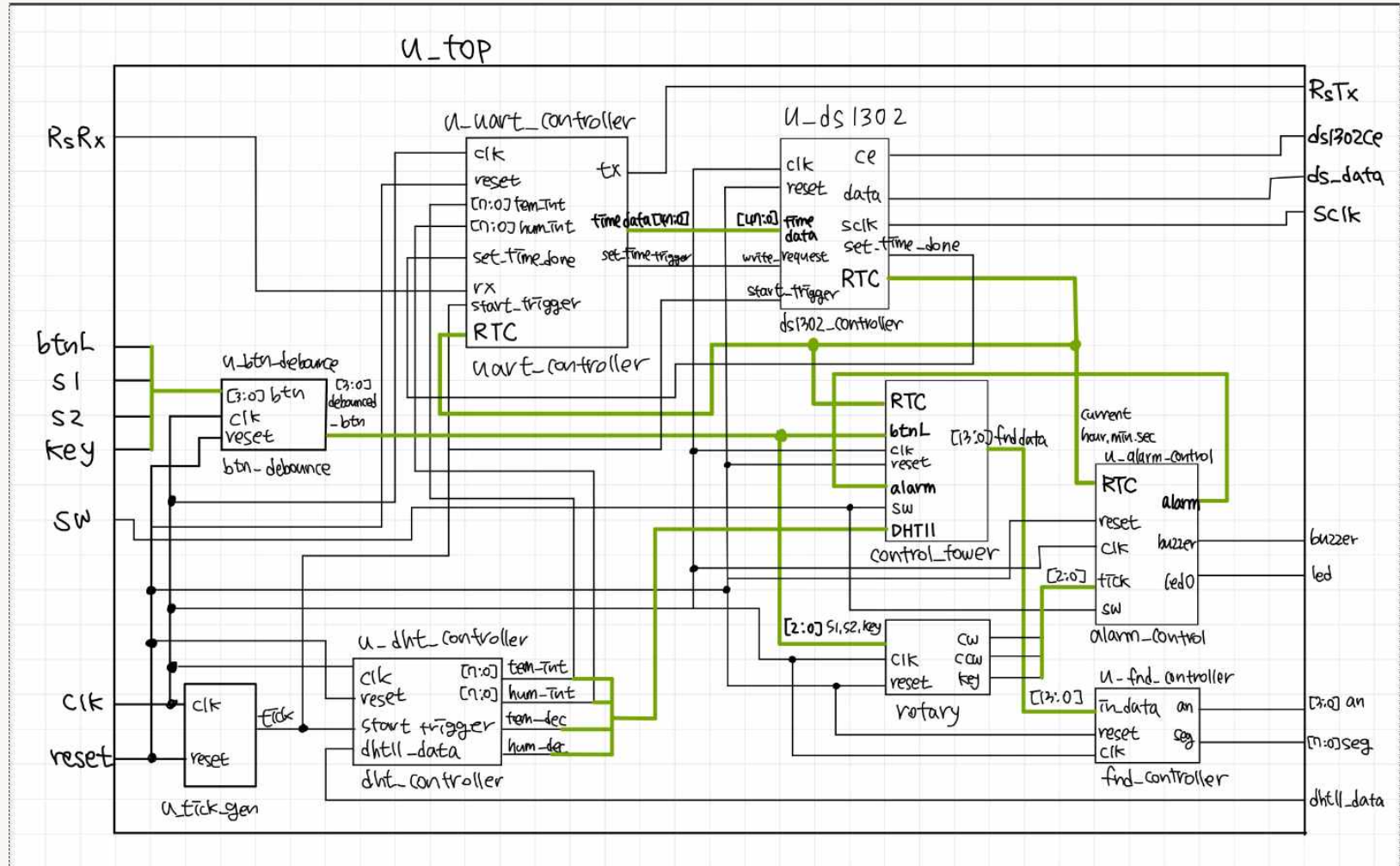
- 시각 정보 FND, UART 출력
- ComPortMaster에서 setrtcyyymmddhhmmss 입력
-> 시각 보정

3) 알람 기능

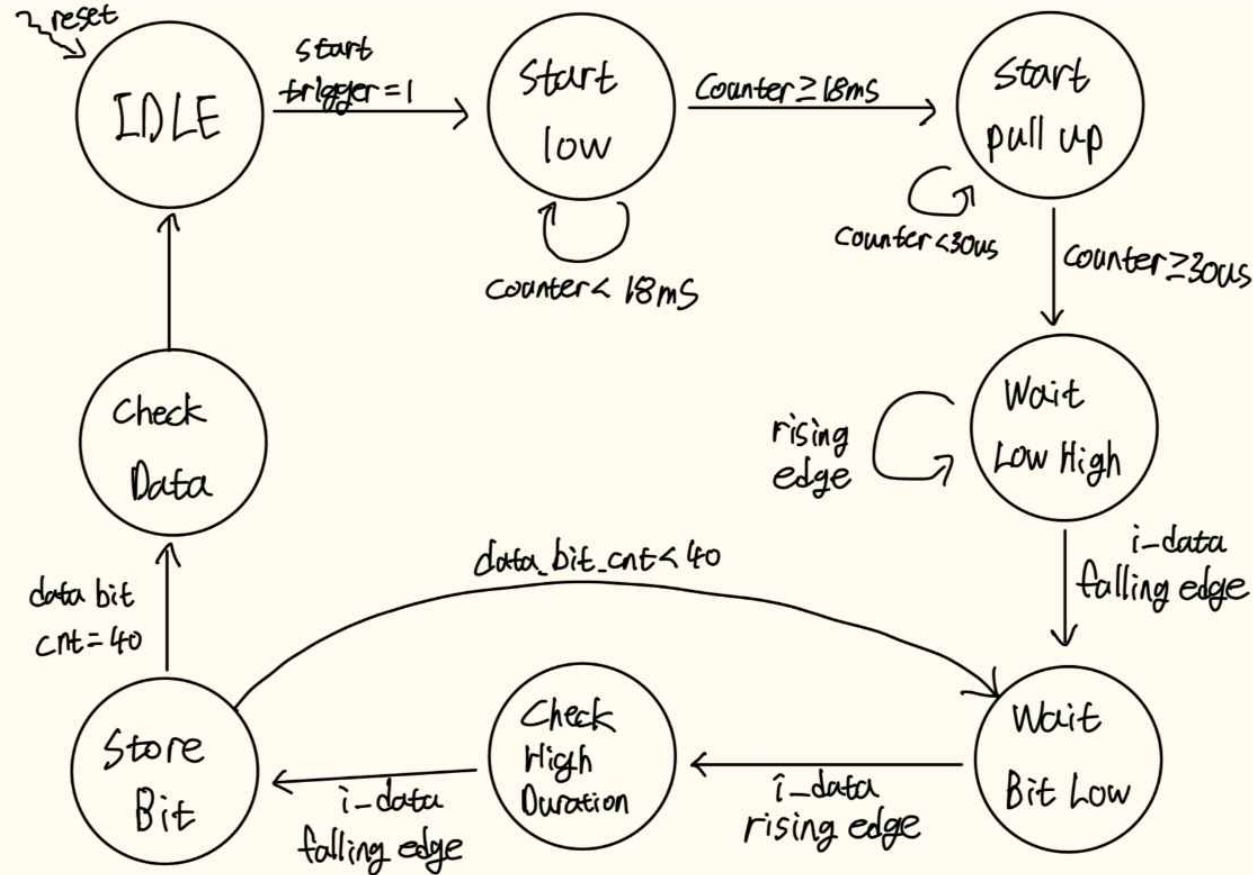
- DS1302 모드일 때 sw0 on 시 알람 설정 모드
- rotary로 시.분 알람 맞추기 -> led0 on
-> ds1302 시각과 맞으면 부저 알림 -> key로 부저 해제

block diagram & FSM

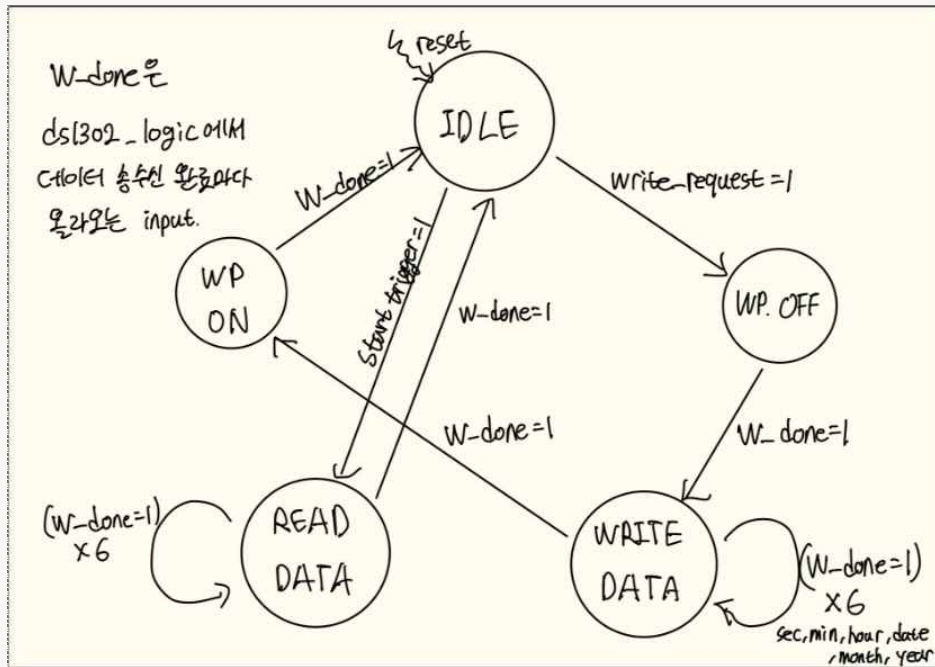
block diagram



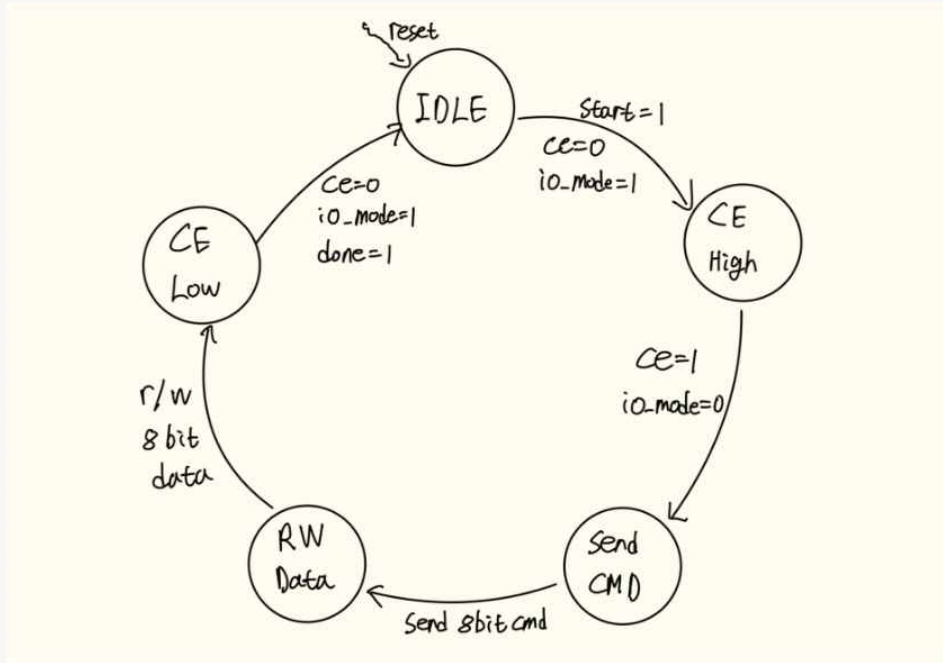
block diagram & FSM - dht11



block diagram & FSM - ds1302



ds1302_controller



ds1302_logic

source code - dht11

```

module dht_controller(
    input clk,
    input reset,
    input start_trigger,
    inout dht11_data,

    output [7:0] hum_int,
    output [7:0] hum_dec,
    output [7:0] tem_int,
    output [7:0] tem_dec
);
    wire w_io_mode, w_o_data, w_i_data;

    dht11 u_dht11(
        .io_mode    (w_io_mode),
        .o_data     (w_o_data),
        .i_data     (w_i_data),
        .dht11_data (dht11_data)
    );

    dht11_main_logic u_dht11_main_logic(
        .clk         (clk),
        .reset       (reset),
        .start_trigger (start_trigger),
        .i_data      (w_i_data),
        .o_data      (w_o_data),
        .io_mode     (w_io_mode),
        .hum_int     (hum_int),
        .hum_dec     (hum_dec),
        .tem_int     (tem_int),
        .tem_dec     (tem_dec)
    );
endmodule

```

```

module dht11(
    input io_mode,
    input o_data,
    output i_data,
    inout dht11_data
);
    assign dht11_data = io_mode ? 1'bz : o_data;
    assign i_data = dht11_data;
endmodule

```

```

localparam IDLE           = 4'd0;
localparam START_LOW      = 4'd1;
localparam START_PULL_UP  = 4'd2;
localparam WAIT_LOW_HIGH  = 4'd3;
localparam WAIT_BIT_LOW   = 4'd4;
localparam CHECK_HIGH_DURATION = 4'd5;
localparam STORE_BIT      = 4'd6;
localparam CHECK_DATA     = 4'd7;

localparam TIME_18mS      = 1_800_000;
localparam TIME_30uS      = 3_000;
localparam TIME_160uS     = 16_000;
localparam TIME_BIT_THRESHOLD = 5_000;

reg [21:0] counter;

reg [3:0] state;
reg i_data_ff1, i_data_ff2;
reg low_high;

reg [15:0] high_time_cnt; // 데이터 high 시간 감지할 counter
reg [39:0] data;
reg [5:0] data_bit_cnt; // 데이터 bit counter
reg recieved_bit;

```

코드 간단 설명

- dht11 data port inout을 io_mode에 따라 i_data, o_data 설정
- state IDLE -> CHECK_DATA
- 각 state에 필요한 TIME parameter 설정
- high_time_cnt : 데이터 input time에서 high 시간 감지할 counter
- 40bit data를 저장할 data reg
- data_bit_cnt : 데이터 bit counter
- recieved_bit : high 타임에 따라 들어온 비트 저장하는 reg

source code - dht11

```
// 2단 FF
always @(posedge clk, posedge reset) begin
    if(reset) begin
        i_data_ff1 <= 1; // 기본 i_data는 idle 상태에서 1
        i_data_ff2 <= 1;
    end else begin
        i_data_ff1 <= i_data;
        i_data_ff2 <= i_data_ff1;
    end
end
end
```

```
// FSM
always @(posedge clk, posedge reset) begin
    if(reset) begin
        state <= IDLE;
        io_mode <= 1; // input mode
        o_data <= 1; // pull_down 18ms를 제외하면 항상 1

        counter <= 0;

        high_time_cnt <= 0;
        data <= 0;
        data_bit_cnt <= 0;
        low_high <= 0;

        hum_int <= 0;
        hum_dec <= 0;
        tem_int <= 0;
        tem_dec <= 0;
    end
end
```

```
case(state)
    IDLE: begin
        io_mode <= 1;
        o_data <= 1;
        high_time_cnt <= 0;
        data <= 0;
        data_bit_cnt <= 0;
        low_high <= 0;
        counter <= 0;

        if(start_trigger) begin
            state <= START_LOW;
        end
    end

    // 18ms start signal low
    START_LOW: begin
        io_mode <= 0; // output mode
        o_data <= 0; // pull_down 동작

        if(counter >= TIME_18mS - 1) begin
            counter <= 0;
            state <= START_PULL_UP;
        end else begin
            counter <= counter + 1;
        end
    end
end
```

코드 간단 설명

- 2단 FF를 통해 setup violation 방지
- io_mode는 1로 초기화된 상태 (input)
- o_data 1로 초기화 -> pull down 18ms 이외에는 항상 1
- IDLE 상태에서 start_trigger가 들어올 경우 state를 START_LOW로 변경 (start_trigger는 1Hz 간격으로 들어옴)
- START_LOW에서 output mode로 변경 및 o_data pull down
- 18ms 지나면 START_PULL_UP

source code - dht11

```

// 30us start signal high
START_PULL_UP: begin
    o_data <= 1;

    if(counter >= TIME_30uS - 1) begin
        counter <= 0;
        state <= WAIT_LOW_HIGH;
    end else begin
        counter <= counter + 1;
    end
end

WAIT_LOW_HIGH: begin
    io_mode <= 1;    // input mode

    // rise, fall edge를 기다림
    if(i_data_ff1 && !i_data_ff2) begin
        low_high <= 1;
    end else if(low_high && !i_data_ff1 && i_data_ff2) begin
        low_high <= 0;
        state <= WAIT_BIT_LOW;
    end
    // 응답 기다림 80us low, 80us high 오류 검출 필요
    // if(counter >= TIME_160uS - 1) begin
    //     counter <= 0;
    //     state <= WAIT_BIT_LOW;
    // end else begin
    //     counter <= counter + 1;
    // end
end

```

```

// data 전송 과정에서 high가 들어오기를 기다림
WAIT_BIT_LOW: begin
    if(i_data_ff1 && !i_data_ff2) begin // 상승엣지를 만나면 state 변경
        state <= CHECK_HIGH_DURATION;
    end
end

// high 시간 측정
CHECK_HIGH_DURATION: begin
    if(!i_data_ff1 && i_data_ff2) begin // 하강엣지일때
        // 50us를 기준으로 1, 0 확인
        if(high_time_cnt >= TIME_BIT_THRESHOLD) begin
            recieved_bit <= 1;
        end else begin
            recieved_bit <= 0;
        end
        state <= STORE_BIT;
    end else begin
        if(high_time_cnt < 16'hffff) begin // 하강엣지 안들어오는 경우 방지
            high_time_cnt <= high_time_cnt + 1;
        end
    end
end

```

코드 간단 설명

- START_PULL_UP에서 o_data pull up 후에 30us 유지 -> WAIT_LOW_HIGH
- WAIT_LOW_HIGH에서 80us low, 80us high 동작을 기다림
- edge까지 wait time을 측정해 일정 duration을 넘기면 오류를 검출하는 개선점 필요
- 상승엣지를 만나면 WAIT_BIT_LOW -> CHECK_HIGH_DURATION
- 하강 엣지를 감지해 50us를 기준으로 1, 0 확인하여 recieved_bit에 저장
- STORE_BIT로 state 이동
- 하강엣지가 들어오지 않을 경우를 방지하기 위해 high_time_cnt 상한선 설정

source code - dht11

```

// data에 bit 저장
STORE_BIT: begin
    high_time_cnt <= 0;
    // shift 시켜가며 밀기
    data <= {recieved_bit, data[39:1]};

    if(data_bit_cnt >= 39) begin
        state <= CHECK_DATA;
    end else begin
        data_bit_cnt <= data_bit_cnt + 1;
        state <= WAIT_BIT_LOW;
    end
end

// check sum error 탐지
CHECK_DATA : begin
    if ((data[7:0] + data[15:8] +
        data[23:16] + data[31:24]) == data[39:32]) begin
        tem_int <= data[7:0];
        tem_dec <= data[15:8];
        hum_int <= data[23:16];
        hum_dec <= data[31:24];
    end else begin
        // error 처리 필요
        hum_int <= 0;
        hum_dec <= 0;
        tem_int <= 0;
        tem_dec <= 0;
    end
    state <= IDLE;
end

default: begin
    state <= IDLE;
end

```

코드 간단 설명

- data를 right shift 이후 저장
- data bit가 40개가 충족되면
CHECK_DATA state로, 그렇지 않다면
data_bit_cnt 1 증가 후
WAIT_BIT_LOW state로 다시 복귀
- CHECK_DATA 에서는 check_sum
error 탐지, 오류 없다면 온습도를 정수
소수로 나누어 각 output reg 에 저장
- check_sum error가 검출될 경우 error
처리 개선점 필요
- state는 다시 IDLE로

source code - ds1302

```

module ds1302_controller(
    input clk,
    input reset,
    input start_trigger,
    input write_request,
    input [7:0] i_sec,
    input [7:0] i_min,
    input [7:0] i_hour,
    input [7:0] i_date,
    input [7:0] i_month,
    input [7:0] i_year,
    output ce,
    output sclk,
    inout ds1302_data,

    output reg set_time_done,
    output reg busy,
    output reg [7:0] o_sec,
    output reg [7:0] o_min,
    output reg [7:0] o_hour,
    output reg [7:0] o_date,
    output reg [7:0] o_month,
    output reg [7:0] o_year
);

```

```

module ds1302(
    input io_mode,
    input o_data,
    output i_data,
    inout ds1302_data
);
    assign ds1302_data = io_mode ? 1'bz : o_data;
    assign i_data = ds1302_data;
endmodule

```

```

ds1302_logic u_ds1302_logic(
    .clk          (clk),
    .reset        (reset),
    .start        (start),
    .i_data       (w_i_data),
    .rw           (rw),
    .cmd          (cmd),
    .wr_data      (wr_data),
    .rd_data      (rd_data),
    .ce           (ce),
    .sclk         (sclk),
    .io_mode      (w_io_mode),
    .o_data       (w_o_data),
    .done         (w_done)
);

```

코드 간단 설명

- start_trigger 는 1Hz 간격의 트리거
- write_request는 set_time 트리거
- set_time_done은 set_time 을 끝냈다는 신호, busy는 ds1302 동작중 신호
- i_(time) 은 set time에 필요한 data, o_(time) 은 read 한 time data
- dht11과 같은 방식의 inout port module
- ds1302_controller -> ds1302_logic으로 start(trigger) rw(read,write 상태) cmd(command) wr_data(write_data) rd_data(read_data) 넘김
- w_done을 받기 전까지 busy 유지

source code - ds1302

```

localparam IDLE      = 4'd0;
localparam WP_OFF    = 4'd1;
localparam WRITE_DATA = 4'd2;
localparam WP_ON     = 4'd3;
localparam READ_DATA  = 4'd4;

localparam READ_SEC   = 8'h81;
localparam READ_MIN   = 8'h83;
localparam READ_HOUR  = 8'h85;
localparam READ_DATE  = 8'h87;
localparam READ_MONTH = 8'h89;
localparam READ_YEAR  = 8'h8D;

localparam WRITE_SEC  = 8'h80;
localparam WRITE_MIN  = 8'h82;
localparam WRITE_HOUR = 8'h84;
localparam WRITE_DATE = 8'h86;
localparam WRITE_MONTH = 8'h88;
localparam WRITE_YEAR = 8'h8C;
localparam WRITE_WP   = 8'h8E;

reg [2:0] state;
reg rw; // rw:0 -> write, 1 -> read
reg start;
reg [7:0] wr_data;
wire [7:0] rd_data;
reg [7:0] cmd;

function [7:0] from_bcd;
    input [7:0] val;
    begin
        from_bcd = ((val[7:4] * 10) + val[3:0]);
    end
endfunction

```

```

always @(posedge clk, posedge reset) begin
    if(reset) begin
        state <= IDLE;
        start <= 0;
        rw <= 0;
        busy <= 0;
        cmd <= 8'h00;
        wr_data <= 8'h00;
    end else begin
        start <= 0;
        set_time_done <= 0;

        case(state)
            IDLE: begin
                busy <= 0;
                if(write_request) begin
                    busy <= 1;
                    rw <= 0;
                    cmd <= WRITE_WP;
                    wr_data <= 8'h00;
                    start <= 1;
                    state <= WP_OFF;
                end else if(start_trigger) begin
                    busy <= 1;
                    rw <= 1;
                    cmd <= READ_SEC;
                    start <= 1;
                    state <= READ_DATA;
                end
            end
        end
    end
end

```

코드 간단 설명

- state : IDLE 상태에서 WP on/off, WRITE_DATA 로 진행되거나 READ_DATA 로 진행되는 두 flow
- R/W parameter 지정
- rw : 0 -> write, 1 -> read
- start는 r/w가 시작될 때 올려주는 trigger
- from_bcd function을 통해 bcd data를 실제값으로 변경
- start, set_time_done은 항상 0으로 유지
- IDLE 상태에서 busy는 항상 0
- write_req or start trigger가 올라오면 cmd, rw, state를 각 flow에 맞게 설정 후 start = 1로 ds1302_logic에 신호 보냄

```

WP_OFF: begin
  if(w_done) begin
    cmd <= WRITE_SEC;
    wr_data <= i_sec & 8'h7F;
    start <= 1;
    state <= WRITE_DATA;
  end
end

WRITE_DATA: begin
  if(w_done) begin
    case(cmd)
      WRITE_SEC: begin
        cmd <= WRITE_MIN;
        wr_data <= i_min;
      end
      WRITE_MIN: begin
        cmd <= WRITE_HOUR;
        wr_data <= i_hour;
      end
      WRITE_HOUR: begin
        cmd <= WRITE_DATE;
        wr_data <= i_date;
      end
      WRITE_DATE: begin
        cmd <= WRITE_MONTH;
        wr_data <= i_month;
      end
      WRITE_MONTH: begin
        cmd <= WRITE_YEAR;
        wr_data <= i_year;
      end
      WRITE_YEAR: begin
        cmd <= WRITE_WP;
        wr_data <= 8'h80;
        state <= WP_ON;
      end
    endcase
    start <= 1;
  end
end

```

```

WP_ON: begin
  if(w_done) begin
    set_time_done <= 1;
    state <= IDLE;
  end
end

READ_DATA: begin
  if(w_done) begin
    case(cmd)
      READ_SEC: begin
        cmd <= READ_MIN;
        o_sec <= from_bcd(rd_data & 8'h7F);
        start <= 1;
      end
      READ_MIN: begin
        cmd <= READ_HOUR;
        o_min <= from_bcd(rd_data & 8'h7F);
        start <= 1;
      end
      READ_HOUR: begin
        cmd <= READ_DATE;
        o_hour <= from_bcd(rd_data & 8'h3F);
        start <= 1;
      end
      READ_DATE: begin
        cmd <= READ_MONTH;
        o_date <= from_bcd(rd_data & 8'h3F);
        start <= 1;
      end
      READ_MONTH: begin
        cmd <= READ_YEAR;
        o_month <= from_bcd(rd_data & 8'h1F);
        start <= 1;
      end
      READ_YEAR: begin
        o_year <= from_bcd(rd_data);
        state <= IDLE;
      end
    endcase
  end
end

default: state <= IDLE;

```

코드 간단 설명

- WP_OFF, WP_ON은 WRITE_DATA 동작 전 후로 WP 0 or 1로 설정해줌
- w_done(logic에서 올라오는 완료 동작) 이 올라오면 각 cmd에 맞는 다음 cmd로 바꾸고, wr_data or o_(time)에 각 값을 넣어줌.
- write flow 도중 WP_ON state에서 set_time_done을 1로 올려 보냄
- i_sec & 0'h7F는 write second에서 7번 비트 ch를 0으로 초기화시키기 위함
- rd_data & 8'h*F는 read data의 의미없는 비트들을 지우기 위함

source code - ds1302_logic

```

localparam IDLE      = 4'd0;
localparam CE_HIGH   = 4'd1;
localparam SEND_CMD  = 4'd2;
localparam RW_DATA   = 4'd3;
localparam CE_LOW    = 4'd4;

localparam DIVIDER    = 50;

reg [2:0] state;
reg [7:0] shifter;
reg [7:0] rd_shifter;
reg [3:0] bit_cnt; // bit 수
reg [21:0] counter;

```

```

always @(posedge clk, posedge reset) begin
    if(reset) begin
        state <= IDLE;
        rd_data <= 8'h00;
        shifter <= 8'h00;
        rd_shifter <= 8'h00;
        done <= 0;
        ce <= 0;
        sclk <= 0;
        io_mode <= 0;
        o_data <= 0;
        bit_cnt <= 0;
        counter <= 0;
    end
end

```

```

done <= 0;
case(state)
    IDLE: begin
        ce <= 0;
        sclk <= 0;
        bit_cnt <= 0;
        io_mode <= 1; // 기본 input mode
        o_data <= 0;

        if(start) begin
            shifter <= cmd;
            rd_shifter <= 8'h00;
            state <= CE_HIGH;
        end
    end

    CE_HIGH: begin
        ce <= 1; // ce를 high (start)
        io_mode <= 0; // 출력모드로
        o_data <= cmd[0];
        state <= SEND_CMD;
    end
end

```

코드 간단 설명

- ds1302_logic state : IDLE -> CE_LOW
- DIVIDER = 50 (1MHz)
- shifter, rd_shifter는 r/w data 저장용
- 기본적으로 io_mode = 1, o_data = 0
- start가 감지될 경우 shifter에 cmd를 저장하고, rd_shifter는 0으로 초기화
- CE_HIGH에서는 ce high 및 출력모드 설정 o_data에 cmd[0]을 미리 뿌려줌
- ds1302는 ce가 올라온 후 sclk에 맞추어 rising edge에서 data값을 수신하는데, 이를 위해 앞으로 이어질 state에서 sclk falling edge마다 미리 cmd data를 o_data에 보냄

source code - ds1302_logic

```

SEND_CMD: begin
    if(counter >= DIVIDER - 1) begin
        counter <= 0;
        sclk <= ~sclk;
        if(sclk) begin // 하강엿지
            shifter <= {1'b0, shifter[7:1]};

            if(bit_cnt >= 7) begin
                bit_cnt <= 0;
                if(rw) begin
                    io_mode <= 1; // 입력모드로
                    rd_shifter <= 8'h00;
                end else begin
                    shifter <= wr_data;
                    io_mode <= 0;
                    o_data <= wr_data[0];
                end
                state <= RW_DATA;
            end else begin
                bit_cnt <= bit_cnt + 1;
                o_data <= shifter[1];
            end
        end
    end else begin
        counter <= counter + 1;
    end
end

```

```

RW_DATA: begin
    if(counter >= DIVIDER - 1) begin
        counter <= 0;
        sclk <= ~sclk;
        if(sclk) begin
            if(rw) begin
                rd_shifter <= {i_data, rd_shifter[7:1]};
            end else begin
                shifter <= {1'b0, shifter[7:1]};
                o_data <= shifter[1];
            end

            if(bit_cnt >= 7) begin
                if(rw)
                    rd_data <= {i_data, rd_shifter[7:1]};
                state <= CE_LOW;
            end else begin
                bit_cnt <= bit_cnt + 1;
            end
        end
    end else begin
        counter <= counter + 1;
    end
end

CE_LOW: begin
    ce <= 0;
    io_mode <= 1;
    done <= 1;
    state <= IDLE;
end

default: begin
    state <= IDLE;
end

```

코드 간단 설명

- SEND_CMD 동안 500ns 마다 sclk 토글하여 1MHz sclk 생성
- sclk가 하강엿지일때 cmd가 담긴 shifter 1 bit right shift 후 o_data 값 변경 -> ds1302에서 다음 상승엿지에 다음 cmd bit 수신 가능
- 8비트가 다 전송되면 rw에 따라 io_mode 변경 shifter 값 변경
- RW_DATA 동안도 1MHz sclk 생성
- 하강엿지마다 read flow 동안은 i_data 값을 읽어 rd_shifter에 저장
- write flow 동안은 o_data를 shifter에 담긴 wr_data로 바꿈
- 8비트 r/w 종료 후 CE_LOW -> IDLE

source code - uart_controller

```

module uart_controller(
    input clk,
    input reset,
    input start_trigger,
    input set_time_done,
    input ds1302_busy,

    input [7:0] hum_int,
    input [7:0] hum_dec,
    input [7:0] tem_int,
    input [7:0] tem_dec,

    input [7:0] rtc_sec,
    input [7:0] rtc_min,
    input [7:0] rtc_hour,
    input [7:0] rtc_date,
    input [7:0] rtc_month,
    input [7:0] rtc_year,

    input rx,
    output tx,
    output [47:0] time_data,
    output set_time_trigger
);

```

```

data_sender u_data_sender(
    .clk          (clk),
    .reset        (reset),
    .start_trigger (start_trigger),

    .hum_int      (hum_int),
    .hum_dec      (hum_dec),
    .tem_int      (tem_int),
    .tem_dec      (tem_dec),

    .rtc_sec      (rtc_sec),
    .rtc_min      (rtc_min),
    .rtc_hour     (rtc_hour),
    .rtc_date     (rtc_date),
    .rtc_month    (rtc_month),
    .rtc_year     (rtc_year),

    .tx_busy      (w_tx_busy),
    .tx_done      (w_tx_done),
    .tx_data      (w_tx_data),
    .tx_start     (w_tx_start)
);

```

```

data_receiver u_data_receiver(
    .clk          (clk),
    .reset        (reset),
    .rx_done      (w_rx_done),
    .rx_data      (w_rx_data),
    .set_time_done (set_time_done),
    .set_time_trigger (set_time_trigger),
    .ds1302_busy  (ds1302_busy),
    .time_data    (time_data)
);

```

코드 간단 설명

- uart_controller에 ds1302에서 올라온 busy, set_time_done 연결
- ds1302에 보낼 time_data 48bit, set_time_trigger output
- uart data_sender 에 공조기, rtc data
- data_receiver 모듈에서 set time 동작 구현 (uart_rx)
- uart_tx, rx 8bit 동작은 기존과 같음

source code - uart data_sender

```
// 숫자 -> ASCII 문자로 변환
wire [7:0] hum_10      = to_ascii_10(hum_int);
wire [7:0] hum_1       = to_ascii_1(hum_int);
wire [7:0] hum_10_dec  = to_ascii_10(hum_dec);
wire [7:0] hum_1_dec   = to_ascii_1(hum_dec);
wire [7:0] tem_10      = to_ascii_10(tem_int);
wire [7:0] tem_1       = to_ascii_1(tem_int);
wire [7:0] tem_10_dec  = to_ascii_10(tem_dec);
wire [7:0] tem_1_dec   = to_ascii_1(tem_dec);

wire [7:0] year_10     = to_ascii_10(rtc_year);
wire [7:0] year_1      = to_ascii_1(rtc_year);
wire [7:0] month_10    = to_ascii_10(rtc_month);
wire [7:0] month_1     = to_ascii_1(rtc_month);
wire [7:0] date_10     = to_ascii_10(rtc_date);
wire [7:0] date_1      = to_ascii_1(rtc_date);
wire [7:0] hour_10     = to_ascii_10(rtc_hour);
wire [7:0] hour_1      = to_ascii_1(rtc_hour);
wire [7:0] min_10      = to_ascii_10(rtc_min);
wire [7:0] min_1       = to_ascii_1(rtc_min);
wire [7:0] sec_10      = to_ascii_10(rtc_sec);
wire [7:0] sec_1       = to_ascii_1(rtc_sec);

localparam IDLE = 2'b00;
localparam SEND = 2'b01;
localparam WAIT = 2'b10;

reg [1:0] state;
reg [4:0] byte_cnt;    // byte 개수
reg print_mode;
```

```
function [7:0] to_ascii_10;
    input [7:0] val;
    begin
        to_ascii_10 = (val / 10) % 10 + 8'h30;
    end
endfunction

function [7:0] to_ascii_1;
    input [7:0] val;
    begin
        to_ascii_1 = val % 10 + 8'h30;
    end
endfunction
```

```
always @(posedge clk or posedge reset) begin
    if (reset) begin
        state <= IDLE;
        tx_start <= 1'b0;
        tx_data <= 8'd0;
        byte_cnt <= 5'd0;
        print_mode <= 0;
    end
end
```

코드 간단 설명

- 8bit data -> ascii 변환 function
- print_mode는 time 데이터 전부 보낸 후 1로 변환 및 공조기 데이터 전송을 위한 register

source code - uart data_sender

```

IDLE: begin
    tx_start <= 1'b0;
    byte_cnt <= 5'd0;
    print_mode <= 0;
    if (start_trigger) begin
        state <= SEND;
    end
end

SEND: begin
    if (!tx_busy) begin
        tx_start <= 1'b1;
        if(!print_mode) begin
            case(byte_cnt)
                5'd0: tx_data <= 8'h32; // 2
                5'd1: tx_data <= 8'h30; // 0
                5'd2: tx_data <= year_10;
                5'd3: tx_data <= year_1;
                5'd4: tx_data <= 8'h2E; // .
                5'd5: tx_data <= month_10;
                5'd6: tx_data <= month_1;
                5'd7: tx_data <= 8'h2E; // .
                5'd8: tx_data <= date_10;
                5'd9: tx_data <= date_1;
                5'd10: tx_data <= 8'h2E; // .
                5'd11: tx_data <= hour_10;
                5'd12: tx_data <= hour_1;
                5'd13: tx_data <= 8'h2E; // .
                5'd14: tx_data <= min_10;
                5'd15: tx_data <= min_1;
                5'd16: tx_data <= 8'h2E; // .
                5'd17: tx_data <= sec_10;
                5'd18: tx_data <= sec_1;
                5'd19: tx_data <= 8'h0A; // LF
                default: tx_data <= 8'h00;
            endcase
        end
    end
end

```

```

        end else begin
            case(byte_cnt)
                5'd0: tx_data <= 8'h54; // T
                5'd1: tx_data <= 8'h3A; // :
                5'd2: tx_data <= tem_10;
                5'd3: tx_data <= tem_1;
                5'd4: tx_data <= 8'h2E; // .
                5'd5: tx_data <= tem_10_dec;
                5'd6: tx_data <= tem_1_dec;
                5'd7: tx_data <= 8'h0A; // \n
                5'd8: tx_data <= 8'h48; // H
                5'd9: tx_data <= 8'h3A; // :
                5'd10: tx_data <= hum_10;
                5'd11: tx_data <= hum_1;
                5'd12: tx_data <= 8'h2E; // .
                5'd13: tx_data <= hum_10_dec;
                5'd14: tx_data <= hum_1_dec;
                5'd15: tx_data <= 8'h0A; // \n
                default: tx_data <= 8'h00;
            endcase
        end
        state <= WAIT;
    end
end

WAIT: begin
    tx_start <= 1'b0; // 1클럭 High 유지 -> Low
    if (tx_done) begin
        if (byte_cnt == (print_mode ? 5'd15 : 5'd19)) begin
            if(print_mode) begin
                state <= IDLE;
            end else begin
                print_mode <= 1;
                byte_cnt <= 5'd0;
                state <= SEND;
            end
        end else begin
            byte_cnt <= byte_cnt + 1;
            state <= SEND; // 다음 글자 전송
        end
    end
end
end

```

코드 간단 설명

- IDLE 에서 초기화, start_trigger(1Hz)에 따라 SEND state로 변경
- tx_busy가 아닌경우 tx_start를 1로 변경 및 print_mode에 따라 uart에 출력할 각 비트에 따른 ascii 값 tx_data에 담기
- WAIT에서 tx_done이 올라올 경우 byte_cnt에 1 증가 하거나 print_mode 변경
- 만약 각 16byte, 20byte를 전부 출력했다면 print_mode 바꾸거나 state를 IDLE로 초기화

source code - uart data receiver

```

if(reset) begin
    for(i=0;i<16;i=i+1) begin
        for(j=0;j<20;j=j+1) begin
            c_queue[i][j] <= 0;
        end
    end
    rear <= 0;
    front <= 0;
    byte_cnt <= 0;
    busy <= 0;
    set_time_trigger <= 0;
    time_data <= 48'd0;
    time_out_cnt <= 0;
end else begin
    set_time_trigger <= 0;
    if(rx_done) begin
        if(rx_data == 8'h0A) begin // 개행문자를 기준으로 cicler queue에 저장
            front <= (front == 4'd15) ? 0 : front + 1; // front 증가
            if(front == rear-1) rear <= rear + 1;
            byte_cnt <= 0;
        end else if(rx_data != 8'h0D) begin
            byte_cnt <= byte_cnt + 1;
            c_queue[front][byte_cnt] <= rx_data; // front 위치에 byte 저장
        end
    end
end

```

```

function [3:0] to_num;
    input [7:0] ascii;
    begin
        to_num = ascii - 8'h30;
    end
endfunction

localparam TIME_OUT_DURATION = 1_000_000; // 10ms timeout duration
reg busy;
reg [7:0] c_queue [15:0][19:0];
reg [3:0] rear, front;
reg [4:0] byte_cnt;
integer i, j;
reg [$clog2(TIME_OUT_DURATION)-1:0] time_out_cnt;

```

코드 간단 설명

- circular queue 사용해서 ds1302가 busy 상태이거나 오류가 발생했을 때 uart_rx data를 저장
- c_queue에 20byte 15개의 방을 지정
- rx_done 이 올라올 경우 rx_data가 개행문자가 아닐경우 front 위치에 byte 저장 및 byte_cnt 증가
- 개행문자일 경우 front 증가, byte_cnt 0

source code - uart data receiver

```

if(!busy) begin
    if(rear != front && !ds1302_busy) begin // set time 데이터가 있다는 뜻
        if({c_queue[rear][0],
            c_queue[rear][1],
            c_queue[rear][2],
            c_queue[rear][3],
            c_queue[rear][4],
            c_queue[rear][5]} == "setrtc") begin
            for(i=0;i<12;i=i+1) begin
                time_data[4*(11-i) +: 4] <= to_num(c_queue[rear][i+6]);
            end
            set_time_trigger <= 1;
            busy <= 1;
        end else begin // set_time command가 아닐 경우
            rear <= rear + 1;
        end
    end
end
end else begin
    if(time_out_cnt >= TIME_OUT_DURATION) begin
        time_out_cnt <= 0;
        busy <= 0;
        time_data <= 48'd0;
        // time out error
    end else begin
        if(set_time_done) begin
            if(rear != front) begin
                rear <= rear + 1; // set한 데이터는 삭제
            end
            time_out_cnt <= 0;
            busy <= 0;
            time_data <= 48'd0;
        end else begin
            time_out_cnt <= time_out_cnt + 1;
        end
    end
end
end
end

```

코드 간단 설명

- busy 상태가 아닐 경우 set time 데이터가 있고, ds1302_busy = 0 일때 time_data를 전부 담은 후 set_time_trigger, busy 올려줌
- 만약 set_time command (setrtc)가 아닐 경우 rear 위치 데이터 삭제
- set_time_done이 올라올 경우 rear 위치 데이터 삭제
- busy 상태인 경우 10ms 동안 set_time_done이 올라오지 않다면 time_out error 발생 -> 개선점

source code - alarm_controller

```

localparam SET_HOUR = 2'd0;
localparam SET_MIN = 2'd1;
localparam SET_DONE = 2'd2;

reg [1:0] set_state; // 위에 3개 상태
reg is_armed; // 알람 울리는지 기억 플래그 (1: 알람 울리기 저장)
reg r_alarm_ring; // 알람 울리는중: 1, 알람 안울림 : 0

localparam DO = 22'd191_112;
localparam TIME_500MS = 26'd50_000_000;

reg [21:0] tone_cnt;
reg [25:0] pattern_cnt;
reg beep_en;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        tone_cnt <= 0; pattern_cnt <= 0; beep_en <= 0; buzzer <= 0;
    end else begin
        if (r_alarm_ring) begin
            if (pattern_cnt >= TIME_500MS - 1) begin
                pattern_cnt <= 0; beep_en <= ~beep_en;
            end else pattern_cnt <= pattern_cnt + 1;

            if (beep_en) begin
                if (tone_cnt >= DO - 1) begin
                    tone_cnt <= 0; buzzer <= ~buzzer;
                end else tone_cnt <= tone_cnt + 1;
            end else buzzer <= 0;
        end else begin
            pattern_cnt <= 0; tone_cnt <= 0; beep_en <= 0; buzzer <= 0;
        end
    end
end
end

```

```

always @(posedge clk or posedge reset) begin
    if (reset) begin
        alarm_hour <= 8'd0;
        alarm_min <= 8'd0;
        set_state <= SET_HOUR;
        is_armed <= 1'b0;
        led0 <= 1'b0;
        r_alarm_ring <= 1'b0;
    end else begin
        if (is_armed && !r_alarm_ring) begin
            if (sw0 && key_tick) begin
                is_armed <= 1'b0;
                led0 <= 1'b0;
                set_state <= SET_HOUR;
            end

            else if (
                (current_hour == alarm_hour) &&
                (current_min == alarm_min) &&
                (current_sec == 8'd0)) begin
                r_alarm_ring <= 1'b1;
                led0 <= 1'b0;
                is_armed <= 1'b0;
            end
        end

        // 알람이 울리고 있을 때
        else if (r_alarm_ring) begin
            // 부저가 울릴 때 sw 상태와 관계없이 로터리 key 누르면 해제
            if (key_tick) begin
                r_alarm_ring <= 1'b0;
                is_armed <= 1'b0;
                led0 <= 1'b0;
                set_state <= SET_HOUR;
            end
        end
    end
end

```

코드 간단 설명

- is_armed : 알람 울리는지 기억 플래그
- r_alarm_ring : 알람 울림 여부
-> 0 : 알람 X, 1: 알람 O
- 부저 생성
-r_alarm_ring == 1일 때
-pattern_cnt(박자 제어) : 500ms 마다 beep_en 신호 켜다
-tone_cnt : beep_en 켜져 있는 동안 'DO' 주파수가 되면 부저 토글되며 소리
- 알람 상태
알람 대기중 일 때 (is_armed == 1 & r_alarm_ring == 0)
- 대기 중에 sw 0 = 1 상태에서 로터리 key 누르면 다 초기화 -> 알람 대기상태 취소
-> Hour 설정으로 state 돌아감
현재 시간 = 알람 시간, 초 0되는 시간
-> r_alarm_ring = 1, is_armed, led = 0
- 알람 울릴 때 (r_alarm_ring == 1)
sw 상태에 무관 -> key 누르면 모두 초기화

source code - alarm_controller

```

// 알람 세팅 모드
else begin
    if (sw0) begin
        case (set_state)
            SET_HOUR: begin
                if (cw_tick) alarm_hour <= (alarm_hour == 23) ? 0 : alarm_hour + 1; // 정방향은 증가
                if (ccw_tick) alarm_hour <= (alarm_hour == 0) ? 23 : alarm_hour - 1; // 역방향은 감소
                if (key_tick) set_state <= SET_MIN;
            end

            SET_MIN: begin
                if (cw_tick) alarm_min <= (alarm_min == 59) ? 0 : alarm_min + 1;
                if (ccw_tick) alarm_min <= (alarm_min == 0) ? 59 : alarm_min - 1;
                if (key_tick) begin
                    set_state <= SET_DONE;
                    is_armed <= 1'b1;
                    led0 <= 1'b1;
                end
            end

            SET_DONE: begin
            end
        endcase
    end else begin
        // 스위치가 내렸을때 다시 시간 세팅으로 돌아감
        set_state <= SET_HOUR;
    end
end
end

```

코드 간단 설명

- 알람 세팅 모드

- SW = 1

-> SET_HOUR

- cw일 때 0~23까지 1씩 증가
- ccw 일때 23~0으로 1씩 감소
- key 누르면 SET_MIN으로 state 변경

-> SET_MIN

- cw일 때 0~59까지 1씩 증가
- ccw 일때 59~0으로 1씩 감소
- key 누르면 SET_DONE으로 state 변경

경

- is_armed = 1, led = 1

-> SET_DONE

- 알람 설정 됐으니 대기

- SW = 0

-> 알람 상태 초기화

-> state가 HOUR 설정으로 돌아감

source code - rotary

```

module rotary(
    input clk,
    input reset,
    input clean_s1,
    input clean_s2,
    input clean_key,

    output reg cw_tick,
    output reg ccw_tick,
    output reg key_tick
);

    reg [1:0] r_prev_state, r_current_state;

    reg r_prev_key;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            r_prev_state <= 2'b11;
            r_current_state <= 2'b11;
            r_prev_key <= 1'b0;
            cw_tick <= 1'b0;
            ccw_tick <= 1'b0;
            key_tick <= 1'b0;
        end else begin
            // 펄스는 1클럭만 유지하고 바로 끄
            cw_tick <= 1'b0;
            ccw_tick <= 1'b0;
            key_tick <= 1'b0;
        end
    end
end

```

```

// 로터리 회전 감지 (상태 변화 분석)
r_prev_state <= r_current_state;
r_current_state <= {clean_s1, clean_s2};

case ({r_prev_state, r_current_state})
    // 시계 방향
    4'b1101, 4'b0100, 4'b0010, 4'b1011 : cw_tick <= 1'b1;
    // 반시계 방향
    4'b1110, 4'b1000, 4'b0001, 4'b0111 : ccw_tick <= 1'b1;
endcase

// key 클릭 감지
r_prev_key <= clean_key;
if (!r_prev_key && clean_key) begin
    key_tick <= 1'b1;
end
end

```

```

end
endmodule

```

코드 간단 설명

- r_prev_state, r_current state
-> 2'b11로 초기화
- cw, ccw, key tick
-> 1클럭 동안만 1(High) 유지
-> 그리고 바로 0으로 초기화
- 위상차 이용하여 로터리 회전 방향 판별
-> 시계 방향
: 11 → 01, 01 → 00, 00 → 10,
10 → 11 순서로 변할 때 cw_tick 발생
- 반시계 방향
: 11 → 10, 10 → 00, 00 → 01,
01 → 11 순서로 변할 때 ccw_tick 발생
- clean_key 입력의 상승 엣지를 감지하여 key_tick 펄스 발생

source code - control_tower

```

module control_tower(
    input clk,
    input reset,
    input btnL,

    input [7:0] hum_int,
    input [7:0] hum_dec,
    input [7:0] tem_int,
    input [7:0] tem_dec,

    input [7:0] rtc_sec,
    input [7:0] rtc_min,
    input [7:0] rtc_hour,
    input [7:0] rtc_date,
    input [7:0] rtc_month,
    input [7:0] rtc_year,

    input sw0,
    input [7:0] alarm_hour,
    input [7:0] alarm_min,

    output reg [13:0] fnd_data
);

    reg r_display_mode; // 0: 온습도 화면, 1: 시간 화면
    reg r_prev;

```

```

// 모드 전환
always @(posedge clk or posedge reset) begin
    if (reset) begin
        r_display_mode <= 1'b0;
        r_prev <= 1'b0;
    end else begin
        if (btnL == 1'b1 && r_prev == 1'b0) begin
            r_display_mode <= ~r_display_mode;
        end
        r_prev <= btnL;
    end
end

always @(*) begin
    if (r_display_mode == 1'b0) begin
        // 온습도 모드
        fnd_data = (tem_int * 100) + hum_int;
    end else begin
        // 시계 모드
        if (sw0 == 1'b1) begin
            // 알람 설정 모드 - sw on
            fnd_data = (alarm_hour * 100) + alarm_min;
        end else begin
            // 일반 시계 모드 - sw off
            fnd_data = (rtc_hour * 100) + rtc_min;
        end
    end
end

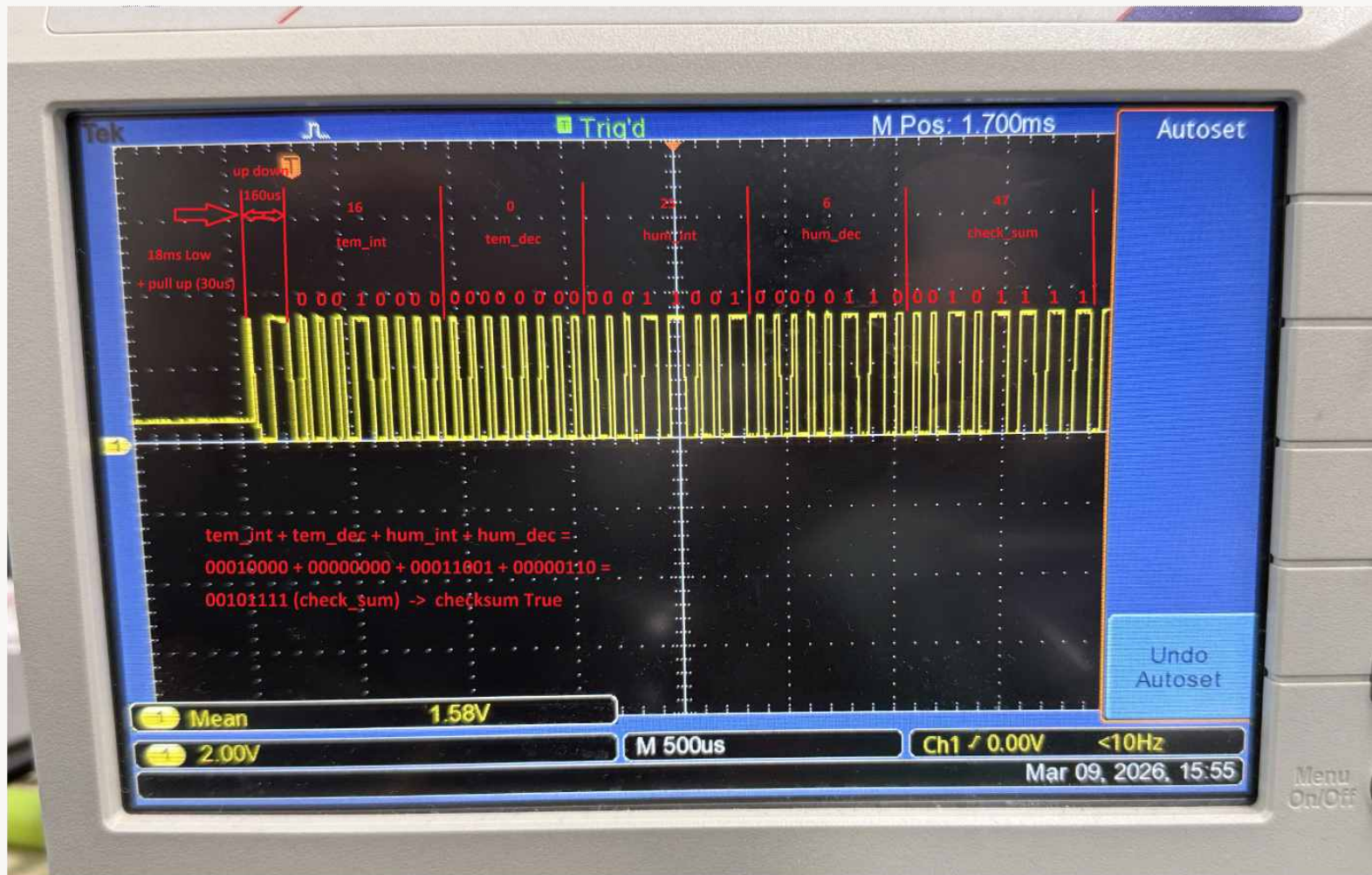
endmodule

```

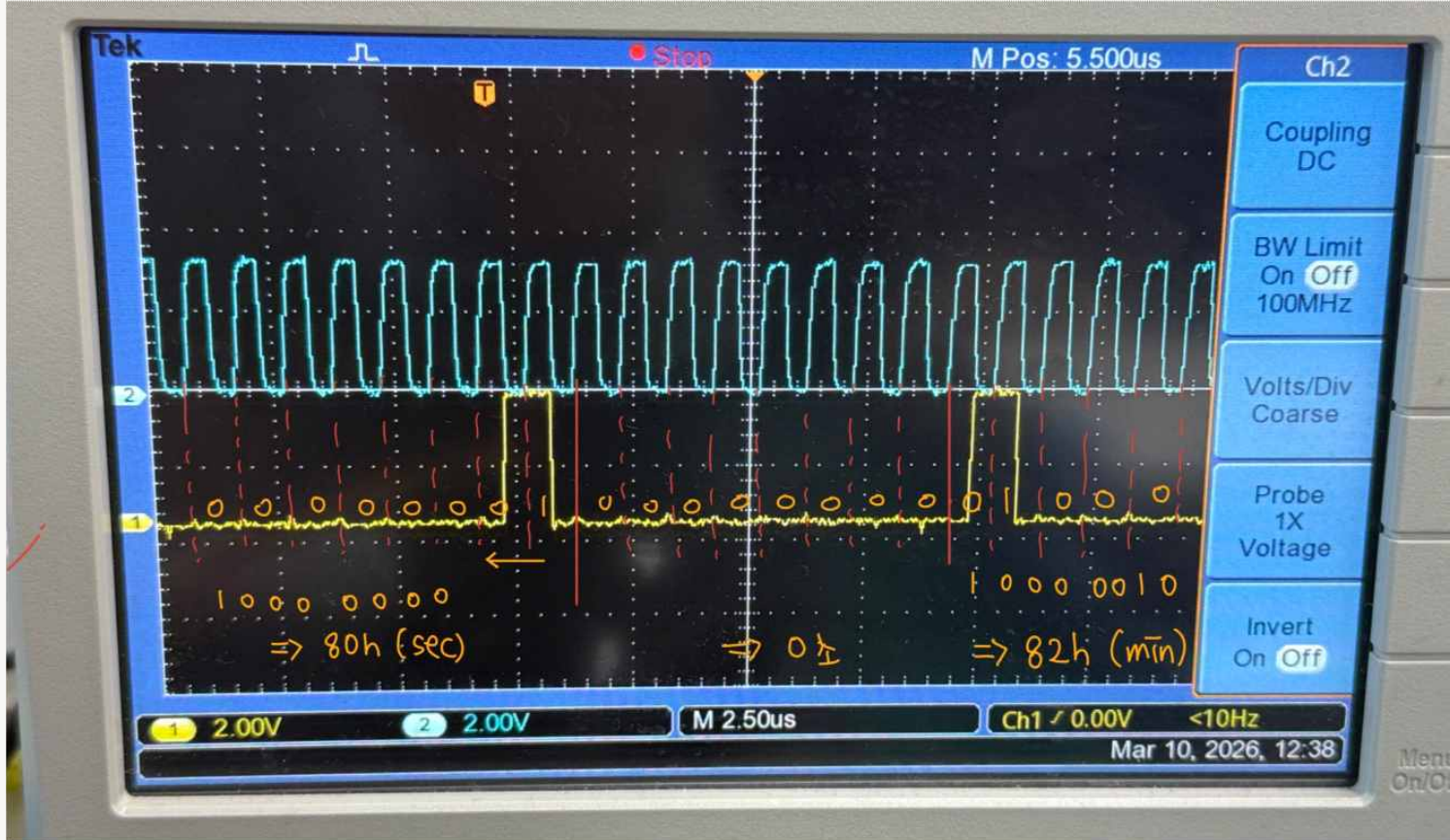
코드 간단 설명

- $r_display_mode = 0$: 온습도 화면
 $r_display_mode = 1$: 시간 화면
- btnL 입력시에 상승 엣지 감지
 -> fnd_display 모드 전환
- 온습도 모드
 -> 온도 * 100 + 습도 출력
- 시계 모드 (sw 상태에 따라 출력 변경)
 - sw on (알람 설정 모드)
 -> 알람 시 * 100 + 알람 분
 - sw off (시계 모드)
 -> 시 * 100 + 분

data 분석 - dht11



data 분석 - ds1302 clk & data 동작 결과



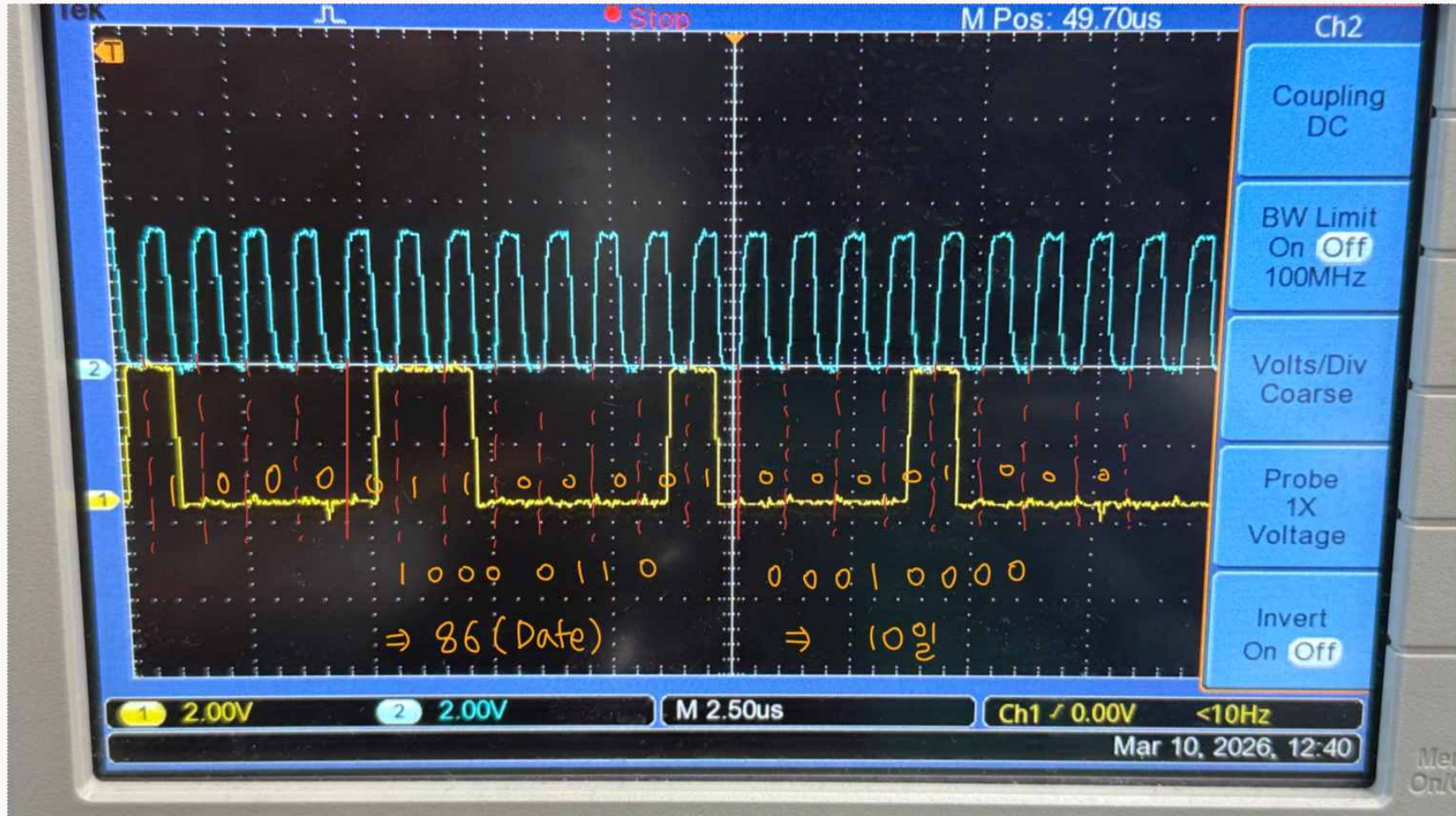
setrtc260310131800

-> 26년 3월 10일 13시 18분 00초

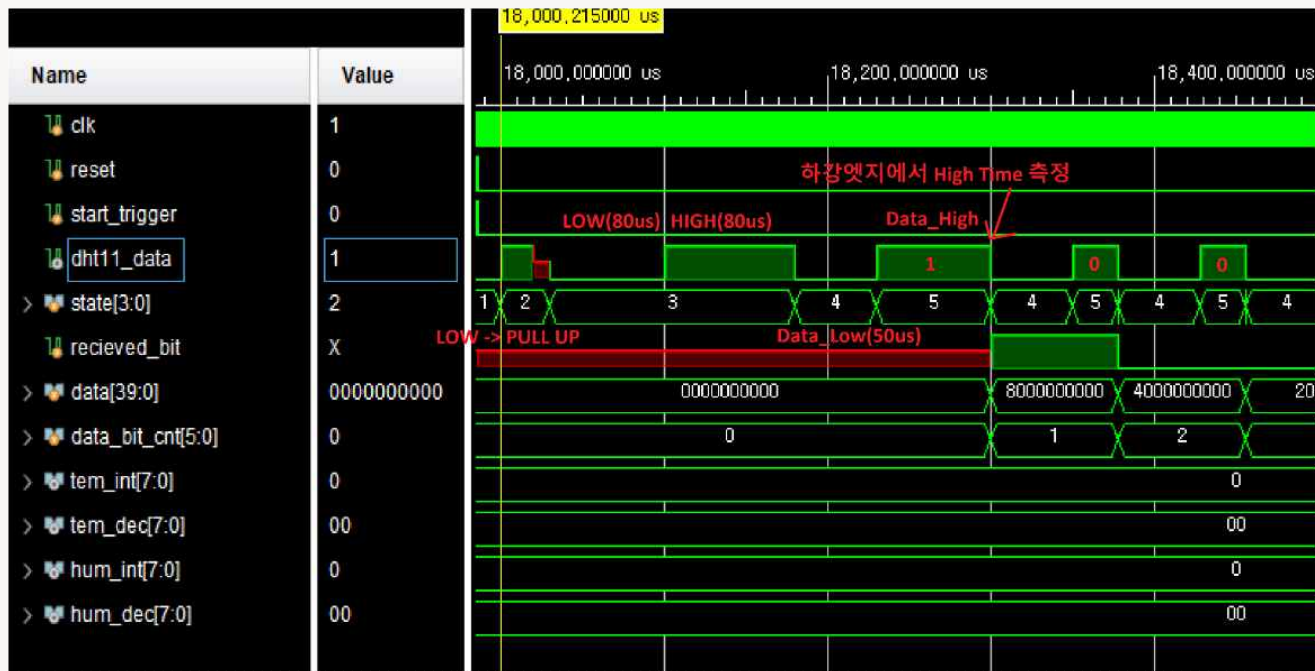
data 분석 - ds1302 clk & data 동작 결과



data 분석 - ds1302 clk & data 동작 결과



data 분석 - dht11 test bench



tem_int [7]	0
tem_int [6]	0
tem_int [5]	0
tem_int [4]	1
tem_int [3]	1
tem_int [2]	0
tem_int [1]	0
tem_int [0]	1

tem_int

hum_int [23]	0
hum_int [22]	0
hum_int [21]	1
hum_int [20]	1
hum_int [19]	1
hum_int [18]	1
hum_int [17]	0
hum_int [16]	0

hum_int

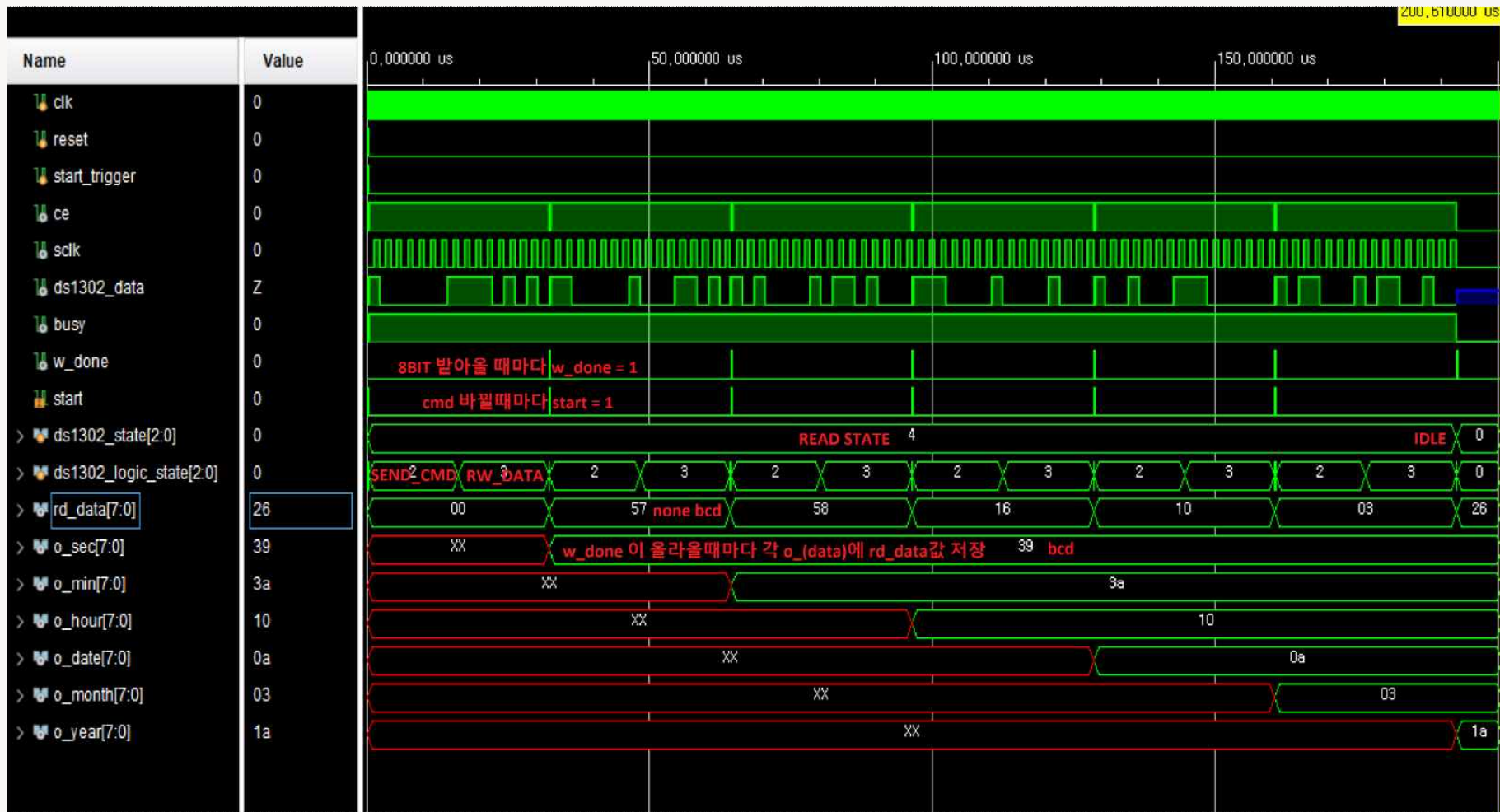
data[39:0]	55003c0019
data [39]	0
data [38]	1
data [37]	0
data [36]	1
data [35]	0
data [34]	1
data [33]	0
data [32]	1

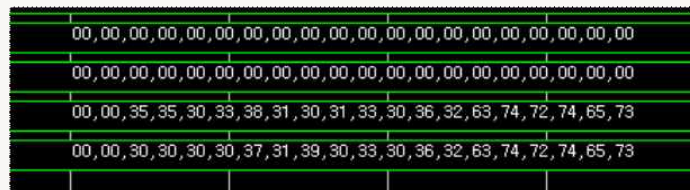
check_sum

tem_int[7:0]	25	25
tem_dec[7:0]	00	00
hum_int[7:0]	60	60
hum_dec[7:0]	00	00

$$\begin{aligned}
 &00011001 (25) + 00000000 + \\
 &00111100 (60) + 00000000 = \\
 &01010101 (85)
 \end{aligned}$$

data 분석 - ds1302 test bench

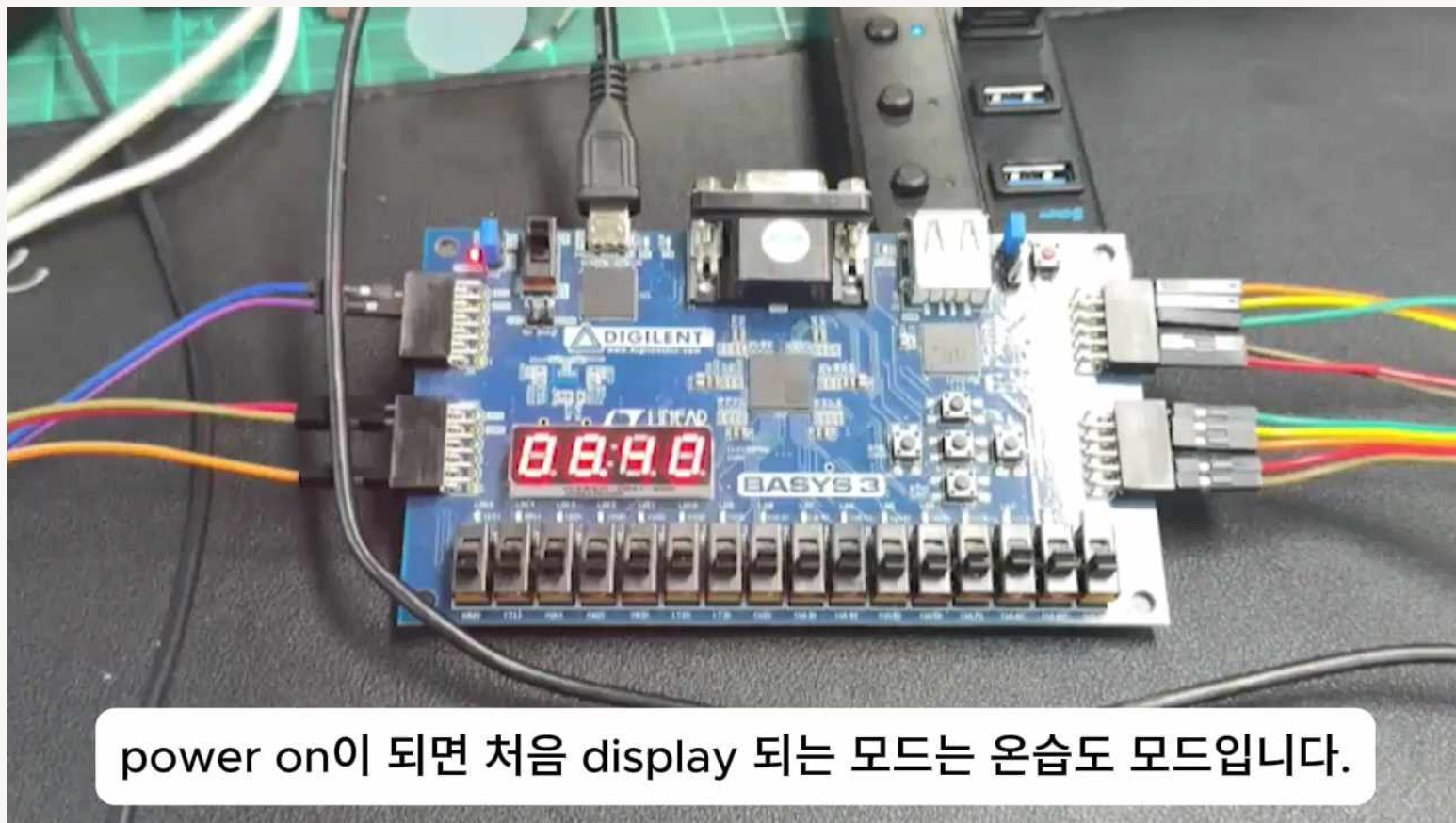




circular queue에 저장된 데이터

data 분석

동작 동영상



power on이 되면 처음 display 되는 모드는 온습도 모드입니다.

결론

- Bolck Diagram, FSM 구성을 먼저 하는것이 프로젝트 규모가 클수록 필수적이라는 것을 느꼈다.
- 동작이 안될 때 디버깅의 방법으로 test bench를 가장 먼저 짜보고, 각 reg들의 값을 보는 방법이 코드의 어떤 부분에서 잘못되었는지 확인하기 편했다.
- test bench의 작동도 잘된다면 oscilloscope로 측정하는 것도 필수적이었다. 하드웨어 문제가 생각보다 잦았다.
- time_out, physical issue 등 에러 발생 가능 부분을 잡아놓고 시간 관계상 error 처리를 하지 못했다. 여러 error를 error controller에 모아 특정 동작을 구현해보고 싶다.

감사합니다.